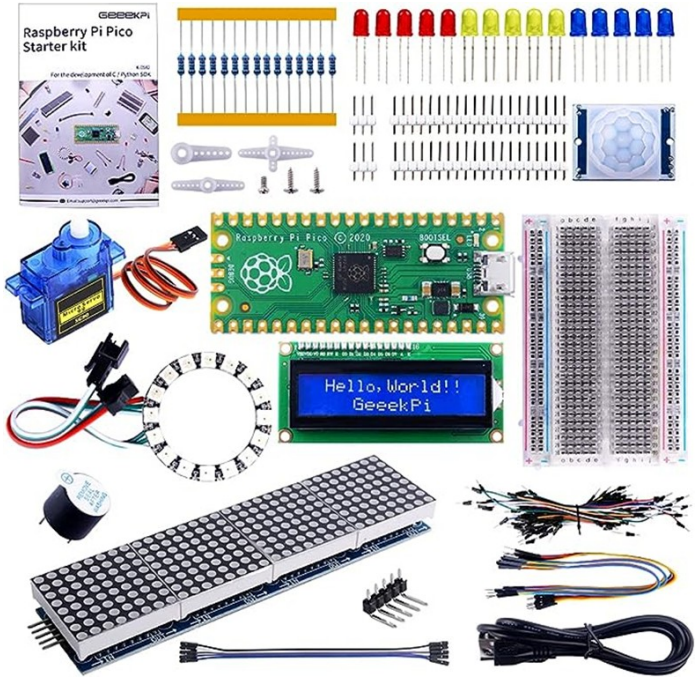
Atelier Raspi

Atelier N°17 La mini serre instrumentée
Module 4 relais

Logo du Raspberry Pico

Logo du MicroPython

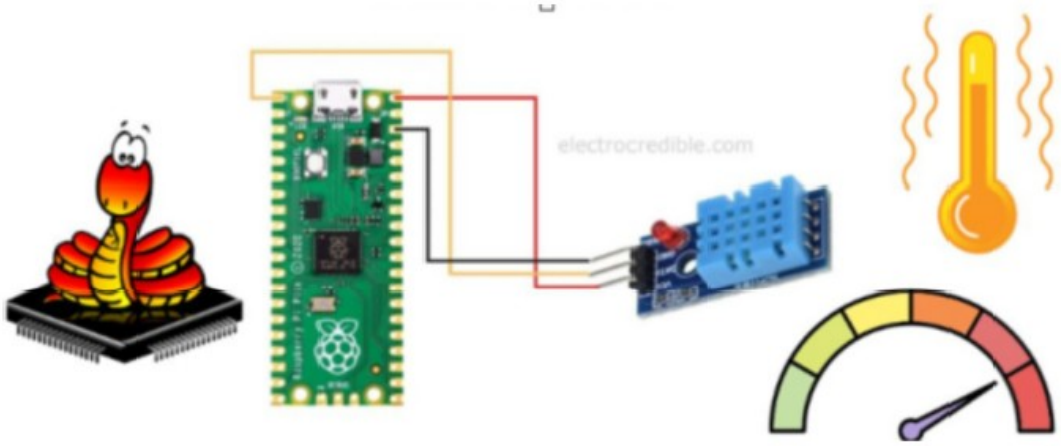


L’atelier a pour valeurs, le partage, l’aide, la formation, le faire et construire ensemble à partir de l’expérience des participants

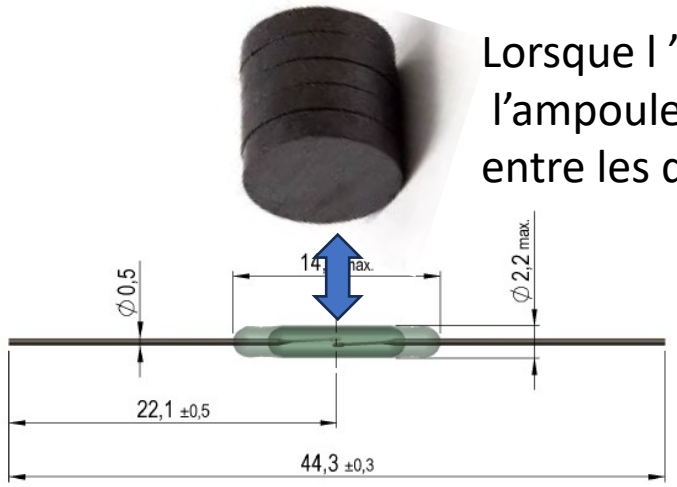
Atelier N°16 La mini serre instrumentée

Module 4 relais

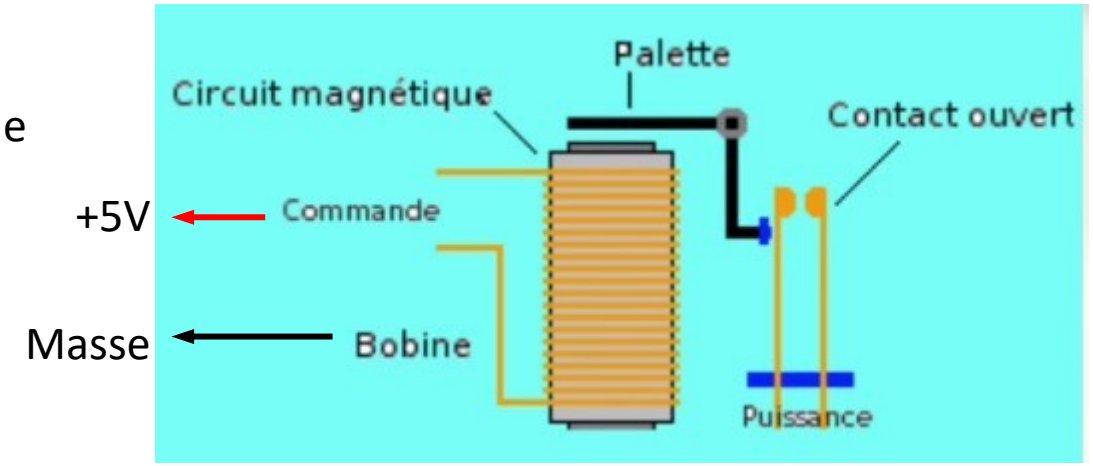
Etape 1 Les relais
Etape 2 Le câblage de la carte 4 relais
Etape 3 les composants du bocal
Etape 4 le brumisateur1/2
Etape 5 le brumisateur2/2
Etape 6 le logiciel
GI1 Glossaire informatique1
GI2 Glossaire informatique2
GI3 Glossaire informatique3
GI4 Glossaire informatique4
GI5 Glossaire informatique5
GI6 Glossaire informatique6
GE1 Glossaire électronique



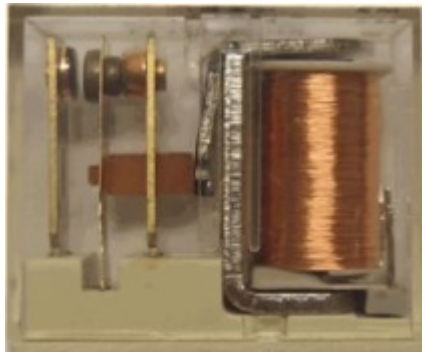
A17E1 Les relais



Lorsque l ’aimant est proche de l’ampoule Reed, le contact se ferme entre les deux lamelles.



Interrupteur de puissance



Vue technologique

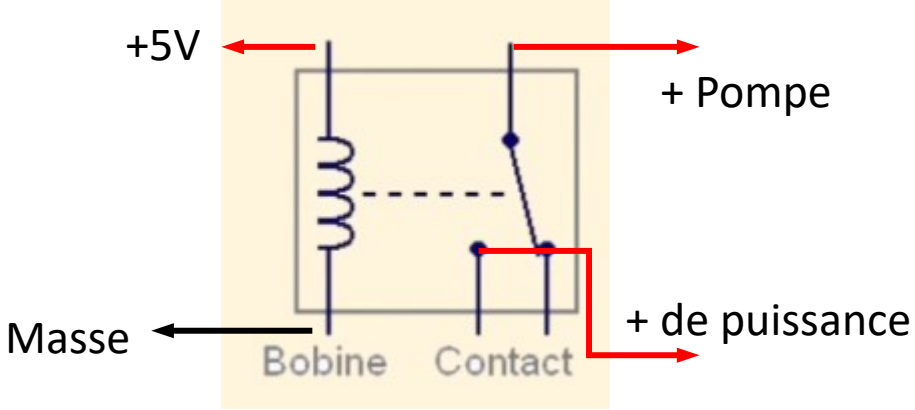
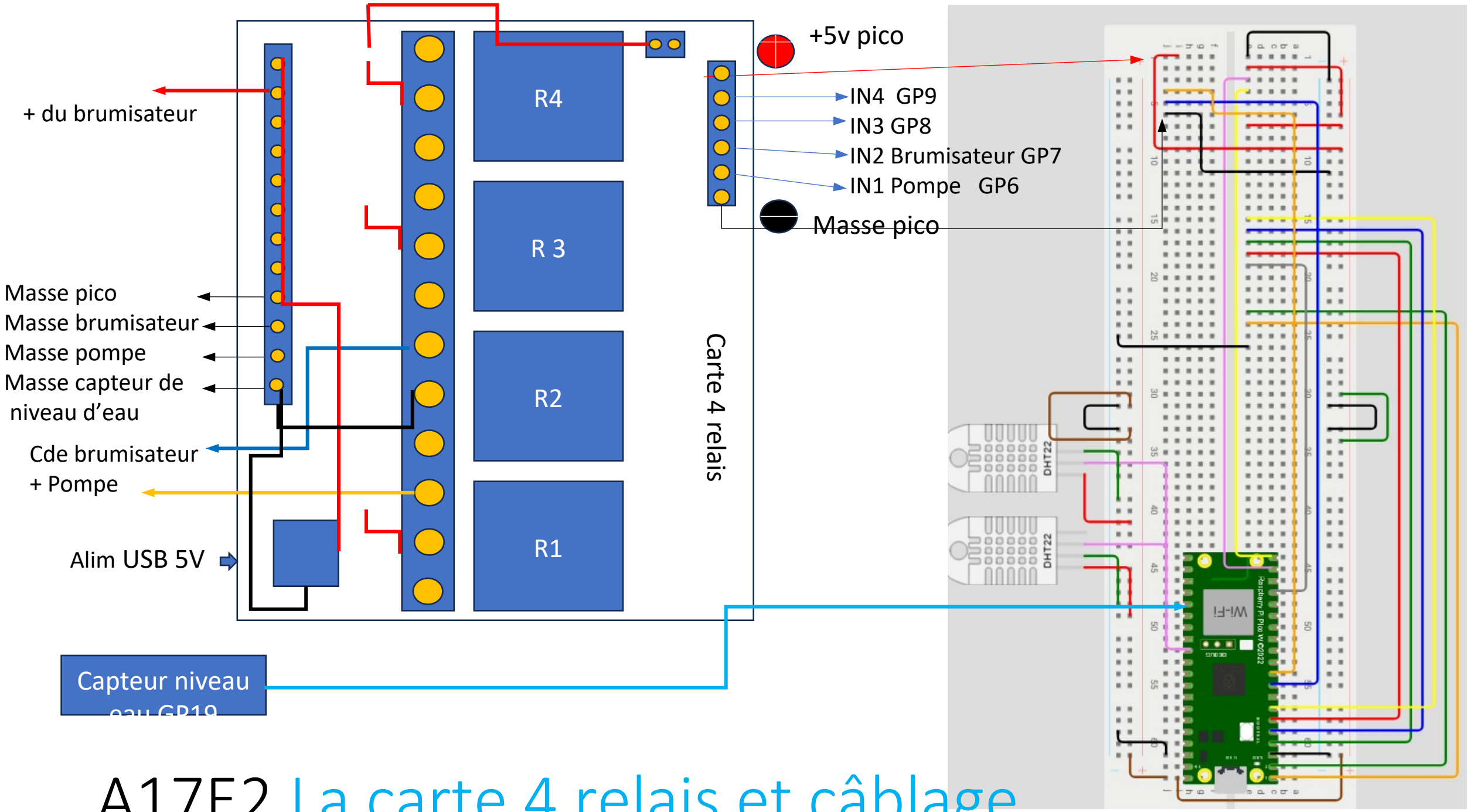
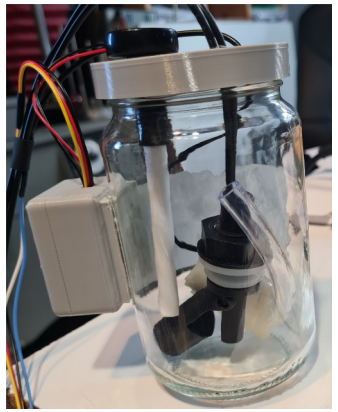


Schéma électrique du relais

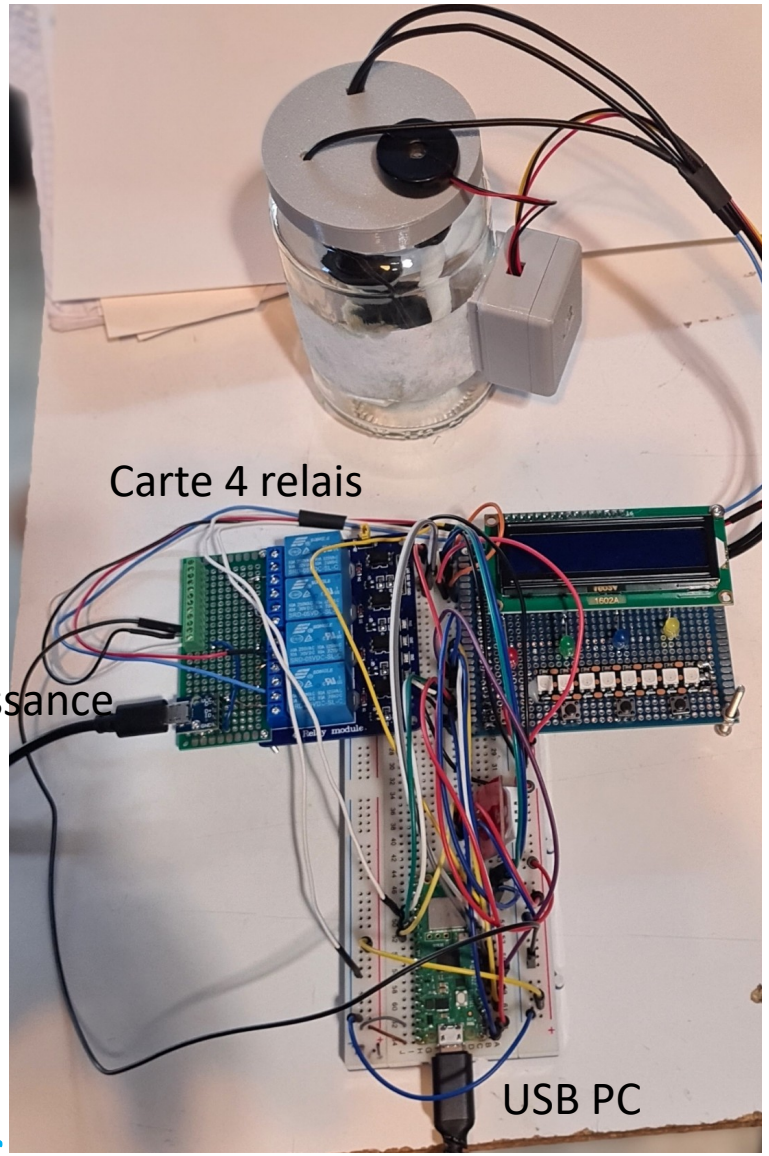


A17E2 La carte 4 relais et câblage



Bocal avec le capteur de niveau d'eau, le brumisateur, la pompe à eau pour arrosage et l'électronique de commande du brumisateur dans la boite grise

Carte pico avec IHM,
carte 4 relais et le bocal



Brumisateur piezzo

Pompe pour arrosage

Capteur de niveau

Avec eau le flotteur remonte

Sans l'eau le flotteur descend

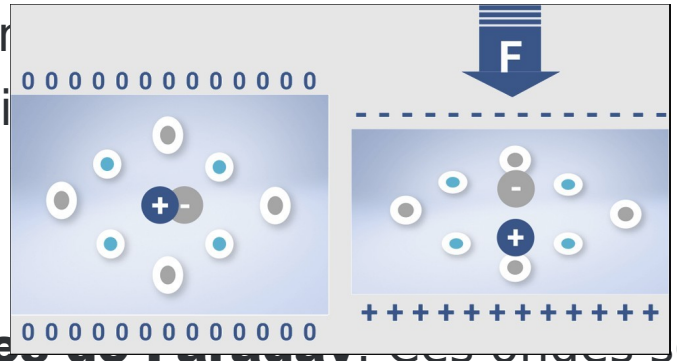
A17E3 Les composants

A17E4 Le brumisateur 1/2

Un **brumisateur piézoélectrique** transforme l'eau en une brume fine grâce à un **cristal piézoélectriq**
à une fréquence ultrasonique. Les principales étapes du processus sont les suivantes :

1. Excitation électrique

Un signal sinusoïdal est appliqué au cristal PZT (Titano-Zirconate de Plor
Sous cette excitation, le cristal se déforme en se contractant et en se di
à la fréquence du signal, créant ainsi une vibration mécanique



2. Génération d'ondes de Faraday

La vibration du cristal, placée sous la surface de l'eau, génère des **ondes de Faraday**. Ces ondes sont
périodiques qui se forment lorsque la fréquence dépasse un seuil critique, provoquant l'instabilité de la

3. Atomisation de l'eau

Ces ondes provoquent un phénomène appelé cavitation : de minuscules bulles de vapeur se forment et implosent rapidement.
L'implosion de ces bulles projette de fines gouttelettes d'eau dans l'air, formant ainsi la brume caractéristique du brumisateur.

Ces ondes de Faraday arrachent des molécules d'eau (et éventuellement de petites quantités d'huiles e
de la surface, les pulvérisant en **gouttelettes très fines** (souvent de l'ordre de quelques microns).

https://phymain.unisciel.fr/le-brumisateur-et-les-ondes-de-faraday/index.html

A17E5 Le brumisateur 2/2

4. Formation de la brume

Les gouttelettes créées restent en suspension dans l'air, formant une **brume sèche** qui recouvre
La brume se disperse doucement, créant un effet visuel et humidifiant l'atmosphère

5. Caractéristiques supplémentaires

•**Fréquence** : typiquement autour de 1,65 MHz, ce qui place le son dans la gamme des ultrasons (
le rend inaudible pour l'humain.

En résumé, le brumisateur piézoélectrique fonctionne grâce à la **vibration ultrasonique** d'un cris
qui crée des ondes de Faraday et pulvérise l'eau en micro-gouttelettes, produisant ainsi une brum

260314 seance avril.py

```python
#L'objectif est de rentrer sur la console de Thonny
#4 messages de max 16 caractères au choix du programmeur,
#puis le programme affiche « prêt » sur l'afficheur lcd
#et suivant l'appui du bouton, on a le message correspondant
#sur l'afficheur.

from machine import I2C, Pin
import time
from quant_debounce_bp import Quant_Debounce_Bp
from lcd1602_i2c import*


# --- LCD ---
i2c = I2C(1, scl=Pin(15), sda=Pin(14), freq=400000)
lcd = I2cLcd(i2c, DEFAULT_I2C_ADDR, 2, 16)
# --- Boutons ---
bp1 = Quant_Debounce_Bp(0, "bp1")
bp2 = Quant_Debounce_Bp(1, "bp2")

lcd.clear()
lcd.move_to(0, 0)
lcd.putstr("Pret...")

print ("....prêt.....")
capteur_niveau=Pin(19,Pin.IN,Pin.PULL_UP)
pompe= Pin(6,Pin.OUT)
brumi= Pin(7,Pin.OUT)
brumipin=Pin(10,Pin.OUT)
pompe.value(1)
brumi.value(1)
brumipin.value(0)
msg1="Pompe ok"
msg2="Brumi ok"
```

A1
7E
5

Le logiciel

```python
# --- Boucle principale ---
while True:
    if capteur_niveau.value()==1:
        if bp1.get_state() == "bp1":
            pompe.value(0)
            lcd.move_to(0,0)
            lcd.putstr(msg1)
            time.sleep(10)
            bp1.bp = None
            pompe.value(1)
            lcd.clear()
            lcd.move_to(0, 0)
            lcd.putstr("Pret...")
        elif bp2.get_state() == "bp2":
            brumi.value(0)
            time.sleep(0.05)
            brumi.value(1)
            lcd.move_to(0,1)
            lcd.putstr(msg2)
            time.sleep(10)
            bp2.bp = None
            lcd.clear()
            brumi.value(0)
            time.sleep(0.1)
            brumi.value(1)
            time.sleep(0.1)
            brumi.value(0)
            time.sleep(0.1)
            brumi.value(1)
            lcd.move_to(0, 0)
            lcd.putstr("Pret...")
        else:
            pass
    else:
        print("ATTENTION, manque d'eau")
        time.sleep(1)
```

GI1 Glossaire informatique1

Instruction du langage : ne nécessite pas d'importer de module pour les utiliser

print('Bonjour') : affiche Bonjour sur la console du PC

While test : Boucle tant que test est VRAI

import : pour importer des modules contenant des fonctions

import time : permet d'appeler les fonctions relatives au temps

time.sleep(x) : arrête l'exécution du programme pendant x secondes

import machine : module contenant les fonctions permettant d'agir sur les périphériques du pi pico

from : pour simplifier l'écriture on importe certaines fonctions directement dans une variable à l'aide de « from »

from machine import Pin : permet d'accéder à la fonction Pin du module machine uniquement.

ma_led_verte = Pin(n, Pin.OUT) : permet d'indiquer comment on souhaite utiliser un GPIO,

ici on a configuré le p I/O N° n en sortie tout ou rien

ma_led_verte.Toggle() : inverse l'état de l'I/O associé à la variable.

Ici l'IO n passera de l'état haut (+3.3v) à l'état bas (0v) ou inversement

: En début de ligne permet d'écrire un commentaire, ce qui est écrit n'est pas une instruction et donc n'est pas pris en compte par le programme.

Variable A=A+1 ->Nouvelle valeur = ancienne valeur+1 , peut s'écrire A+=1

print (‘’ texte ’’, A), **input** (‘’texte’’, B)

Opérateurs: +,-,*,/,%, Données: int(), float(), str(), bool()=True or False

GI2 Glossaire informatique2

Liste -> Création d'une liste, liste_de_noms =['Toto', 'Hubert', 'Jacques', 'Philippe']
liste_quelconque = ['Toto', 1, 'Philippe', 4,5,6]
print ('Nombre d'articles dans la liste : ', len(liste_quelconque))
for items in liste_quelconque:
  print (items) -> parcours la liste et imprime chaque item de la liste
Pour récupérer un élément de la liste, **variable= liste[i],** i étant l'indice d'ordre dans la liste

Dictionnaire-> liste d'éléments sous la forme d'une paire key:valeur.
N = {'voiture':'4 roues', 'moto':'2 roues'}
On accède à la valeur d'un key en l'utilisant comme index (clé de recherche):
print(N['voiture']) -> '4 roues'

Fonction-> **def nom_de_ma_fonction(args) :**
on peut ensuite l'appeler dans un programme avec: **nom_de_ma_fonction()**

Module-> from machine import Pin -> importe la fonction Pin du module machine
P0 = Pin(0, Pin.OUT) -> on paramètre le GPIO 0 en sortie
P0.value(1) -> on lui donne la valeur 1 ou état « haut »

GI3 Glossaire informatique 3

Boucles

for degree in range(0,180,1):

XXXXX -> la variable degree vat varier de 0 à 180 avec un pas de 1, pour chaque valeur de cette variable, XXXX sera exécuté

global ma_variable -> permet d'avoir accès à ma_variable du programme principal dans la def d'une fonction

```
def ma_fonction():
    global ma_variable
    print( ma_variable)
    ma_variable += 2
    return ma_variable
```

Return ma_variable -> permet de donner au reste du programme la nouvelle valeur de ma_variable ou d'une nouvelle variable si creée dans la fonction

GI4 Les fonctions du module time

time.ticks_ms() -> donne le temps présent en ms
time.ticks_diff(ticks1, ticks2) -> donne la différence entre ticks1 et 2

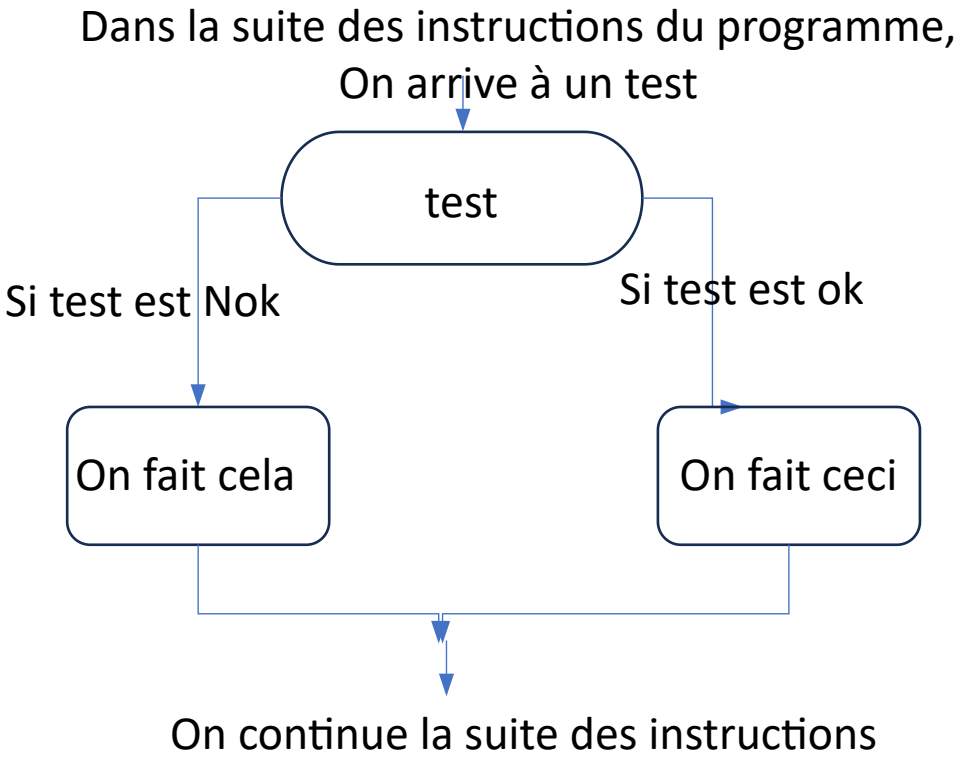
time.sleep (2) -> met un délai de 2 secondes

utime travaille comme time mais en ms
utime.sleep (100) -> met un délai de 100ms

GI5 Glossaire informatique 3

Les tests:

l'instruction **if…else** permet d'exécuter un bloc de code en fonction de la véracité d'une condition,
où le corps de l'instruction **if** s'exécute si la condition est **vraie**,
et le corps de l'instruction **else** s'exécute si elle est **fausse**,
avec une indentation pour définir les blocs de code.



Exemple:
Dans le programme A=2

```
If A >10:
    print('A supérieur à 10')
else:
    print ('A inférieur à 10')
```

On continue d'exécuter les instructionsqui suive

GI6 Les fonctions du module PWM

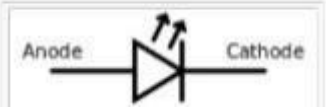
Fonctions utilisées:

servoPinPan **= PWM(Pin(16))** # servoPinPan est un objet, initialise le Pin GPIO 16 en sortie PWM
servoPinPan.**freq(50)** # Initialise la fréquence des signaux carrés à 50Hz soit 20ms
servoPinPan**.duty_u16(int(newDuty))** # duty_u16 est le rapport cyclique entre le temps du signal carré haut sur sa période, new duty est une variable, le max est à 65535.

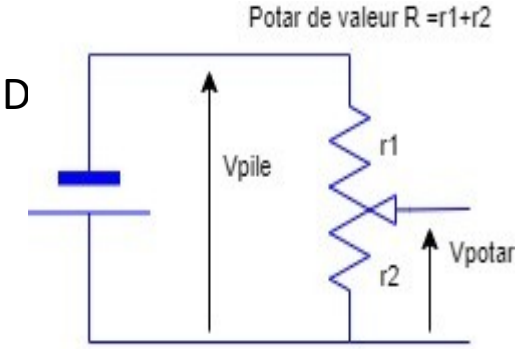
servoPinPan**.deinit()** # réinitialise le PWM

servoX**.duty_ns(**pulseDX) #commande le rapport cyclique du PWM en nanosecondes suivant la variable **pulseDX** de l'objet **servoX**

GE1 Glossaire électronique

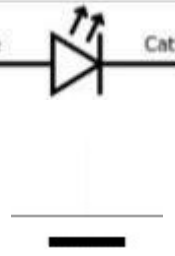


Symbole de la diode LED

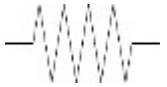


Vpotar=Vpile* r2/(r1+r2)

Couleurs	Tension de seuil ou Vf	If (mA)	Longueur d' onde
Rouge	1,6 V à 2 V	6à20	650 à 660 nm
Jaune	1,8 V à 2 V	6à20	565 à 570 nm
Vert	1,8 V à 2 V	6à20	585 à 590 nm
Bleu	2,7 V à 3,2 V	6à20	470 nm
blanc	3,5 v à 3,8 v	30	



Pile



Résistance

BP

Condensateur
de filtrage signal

Condensateur
Chimique, réserve d'énergie

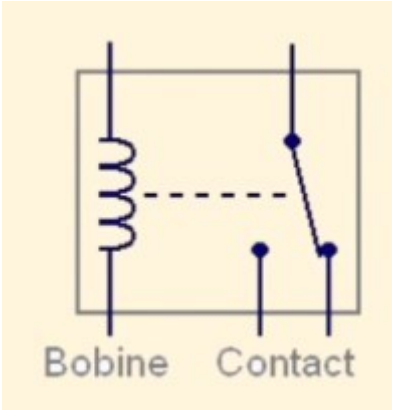
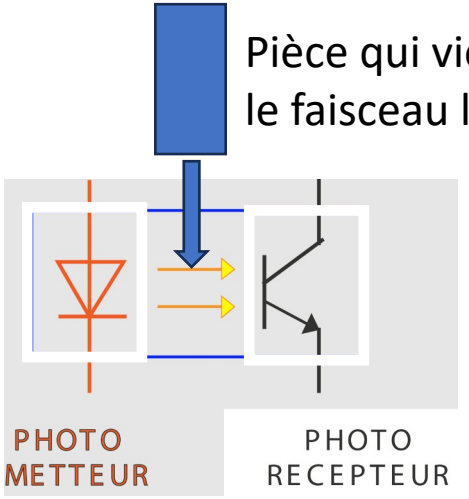


Schéma électrique du relais



Pièce qui vient couper
le faisceau lumineux IR